Arrays I

```
const users = []; // empty array
const grades = [10, 8, 13, 15]; // array of numbers
const attendees = ["Sam", "Alex"]; // array of strings
const values = [10, false, "John"]; // mixed
```

.length property

```
[].length; // 0

const grades = [10, 8, 13, 15];
grades.length; // 4
```

Get element by index

```
const users = ["Sam", "Alex", "Charley"];
users[1]; //"Alex"

const users = ["Sam", "Alex", "Charley", "Blane", "Crane"];
users.at(0); //"Sam"
users.at(1); //"Alex"
users.at(-2); //"Blane"
users.at(-1); //"Crane"
```

Adding an element

You can add an element to the end of an array using the `.push()` method.

```
const numbers = [10, 8, 13, 15];
numbers.push(20); // returns 5 (the new length of the array)
console.log(numbers); // [10, 8, 13, 15, 20];
```

Array iteration is one of the most important concepts that you will use in JavaScript

```
const grades = [10, 8, 13];

grades.forEach(function(grade) {
    // do something with individual grade
    console.log(grade);
});

grades.forEach(insert_callback_here);
```

A callback is a function definition passed as a parameter to another function. In our example above, here's the callback function:

```
const grades = [10, 14, 15]; // array (plural)
grades.forEach(function(grade) { // array item (singular)
    console.log(grade);
});


const people = ["Sam", "Alex"]; // array (plural)
```

```
people.forEach(function(person) { // array item (singular)
    console.log(person);
});
```

Returning from loop

```
function logUserIds(userIds) {
    userIds.forEach(function(userId) {
        console.log(userId);
    });
    return true; // ✅ return from the logUserIds function
}
```

Show previous notes